

Donghyung Lee

DevOps Engineer | Career Description

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- IaC-based infrastructure automation and version control using Terraform (100% GCP resource codification)
- On-premises to cloud hybrid architecture design and migration experience

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design and enhancement using GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction with 3x weekly deployments shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Jenkins-based Unity Android/iOS build automation and Slack-integrated deployment notifications

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Integrated metric/log monitoring with PLG Stack (Prometheus, Loki, Grafana)
- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- 60-70% MTTR improvement, SLI/SLO-based 99.9% service availability achieved

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- 90%+ manual work reduction through data collection, deployment, and document management automation
- Internal LLM service (Ollama + Open WebUI) ensuring 100% internal security guideline compliance and dev team productivity improvement
- Internal tool development: GitLab Webhook doc sync, Slack Bot APK deployment notifications

Kubernetes Platform Modernization & Open-Source Contribution

Zero-downtime migration of network/logging stacks and releasing reusable public charts as open source

- Migrated 11 ingress-nginx instances to a single NGINX Gateway Fabric control-plane + 11 Gateway CRs with zero downtime (11/11 cutover, wildcard TLS consolidated)
- Migrated the ECK Operator-based logging stack from a local chart to an OCI chart with zero downtime (CR/PVC names preserved, cluster_uuid unchanged, no data loss)
- Released 4 open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck, certmanager-letsencrypt) and multiple composite GitHub Actions on ArtifactHub / GitHub Marketplace

Data Engineering & AI/ML Infrastructure

End-to-end ownership from GCP data pipelines to in-house LLM services

- Built a multi-source (MongoDB · CloudSQL · GA · Dune) → BigQuery → Google Sheets pipeline by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler — entire infrastructure 100% Terraform IaC
- Deployed an internal LLM service (Ollama + Open WebUI) on a Kubernetes-hosted on-premises GPU server (RTX 3060), saving ~\$100/month by replacing external AI SaaS subscriptions and ensuring 100% internal security compliance
- 95% data-collection automation and 0% data gap rate by eliminating human error

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As a dedicated DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture.

Currently providing secondary technical support and operations for the live game 'Souls' in collaboration with the publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of \$60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud elasticity unnecessary, while only QA/Production required scalability and stability. Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.

Both environments operate independently without VPN connections, with clearly separated roles to prevent failure propagation. Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version. Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- Intermittent timeouts during ClusterIP service communication in Kubernetes IPVS mode. Resolved through Cilium eBPF packet flow analysis and IPVS conntrack threshold tuning, and built a Cilium Network Policy-based zero-trust security model to enhance inter-pod traffic visibility and security
- Images built on-prem (Harbor) and images pushed separately into AWS (ECR) ended up living on only one side, forcing frequent cross-cluster push/pull through the public internet and pushing up build time, cost, and failure rate. Automated bidirectional sync — Harbor's native event-based Replication for Harbor → ECR (auto-replicate on every push), and an EventBridge → SNS → GitHub Actions workflow that runs `skopeo copy` for ECR → Harbor — cutting deployment time by 40%
- During rolling deploys, the ALB target deregistration delay (lowered from the default 300s to 60s by operational policy) didn't line up with Kubernetes Pod SIGTERM handling, causing some session connection drops. (1) Added a 60s sleep in the Pod lifecycle.preStop hook so the readiness probe fails first — making ALB mark the target unhealthy and stop sending new requests; (2) raised terminationGracePeriodSeconds to 90s (ALB drain 60s + in-flight handling 30s); (3) implemented a SIGTERM handler in the game-server container that waits for in-flight matches to finish — achieving zero-downtime deploys with 0 game-session connection drops

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% (\$60,000 → \$48,000), reinvesting ~\$144,000 annual savings into new project infrastructure for resource reinvestment cycle
- Offloaded dev servers to on-premises, cutting ~\$1,000/month in AWS resource costs and optimizing resource usage
- Ensured operational stability and prevented failure propagation through environment role separation

Tech Stack

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. Developers simply Git Push to trigger automatic build→test→deploy, with ArgoCD monitoring Kubernetes cluster state for declarative deployments.

Eliminated Docker-in-Docker dependency using Kaniko for Docker image builds, and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- When `syncPolicy.automated.selfHeal=true` was active on an ArgoCD Application, the controller immediately reverted operators' temporary hot-fixes, making incident debugging difficult. Split the policy by environment — `prod: selfHeal=false` with manual sync windows; `dev/staging: selfHeal=true` — and integrated Slack notifications for every sync event to gain both change traceability and stability
- Package installation failure on CI Runner due to GPG key expiration after GitLab upgrade. Automated GPG key renewal process to prevent recurrence
- Build time spikes due to cache invalidation during Kaniko builds. Stabilized build times by combining `-cache=true --cache-ttl=24h --snapshot-mode=redo` options to improve cache hit rate

Results

- 67% deployment time reduction (30min → 10min)
- 90% manual work automation (build, deploy, config apply)
- Deployment automation increased weekly deployments 3x+ (1-2 → 5-7), shortening Time to Market for new features, while deployment resource efficiency enabled the dev team to focus on core feature development

Tech Stack

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

Project 3: Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement)

Contribution 100% (System design & implementation) 2025.06 ~ Present (Operations & Enhancement)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents, with no unified monitoring system in place.

Approach

Phase 1 (ELK Setup): Built a centralized logging system using Elasticsearch, Logstash, Kibana, and Filebeat. Collected logs from all servers and containers via Filebeat, processed JSON parsing and field mapping in Logstash, stored in Elasticsearch, enabling real-time monitoring and search through Kibana dashboards.

Phase 2 (EFK Migration): Migrated to Fluent Bit + Fluentd based EFK Stack due to Logstash's high JVM memory usage (1GB+), Filebeat's Helm ecosystem mismatch, and log pipeline custom extensibility limitations. C-based lightweight collector Fluent Bit runs as DaemonSet on each node with automatic Kubernetes metadata tagging, while Fluentd handles log processing/filtering with multi-output support before forwarding to Elasticsearch. Developed a chart upgrade automation script to preserve custom PVC templates during upstream Helm chart auto-upgrades.

Phase 3 (ECK Operator Migration + Stack 9.0 Major Upgrade): With Elastic's official Helm Chart stalled for over two years and CRD-based declarative management (ECK) becoming mainstream, performed a major upgrade of Elastic Stack from 8.5.1 to 9.0.0 and introduced ECK Operator 3.3.2 in the `elastic-system` namespace.

Redefined Elasticsearch/Kibana as Custom Resources managed via a local chart (CR wrapper) to align with repo convention, and adopted a parallel deployment (`monitoring` → `logging ns`) and cutover strategy for zero-downtime migration.

Under the ECK-based model, established automatic TLS certificate issuance/rotation (1y / 30d), elastic account password management via Helm templates, and a rolling upgrade model completed by a single-line CR spec.version change — eliminating the manual operational burden.

Troubleshooting

- Logstash parsing failures due to null bytes (`\x00`) in game server logs. Resolved by adding a preprocessing step using `gsub` in mutate filter to remove null bytes
- Field mapping conflicts (`mapper_parsing_exception`) due to different data types for same field names in Elasticsearch indices. Resolved by defining explicit mappings in index templates and applying type conversion filters in Logstash
- Elasticsearch indices switched to read-only mode due to disk watermark exceeded during log volume spikes. Applied ILM (Index Lifecycle Management) policies to auto-delete indices older than 7 days and stabilize disk usage
- [EFK] Fluent Bit WAL file write failures on NFS volumes. Resolved by injecting custom PVC volume patch into upstream Helm chart's `_pod.tpl` and configuring the upgrade script to auto-preserve during upgrades
- [EFK] Log loss due to Fluentd output buffer overflow. Resolved by tuning `chunk_limit_size` and `flush_interval`, setting `overflow_action` to `block` to prevent log loss
- [ECK] Kibana 9 enforces the `Secure` flag on session cookies when `server.ssl.enabled=true`, causing browsers to drop cookies and fail login in HTTP environments. Resolved with the `tls.disabled + publicBaseUrl http:// + xpack.security.secureCookies:false` combination to restore HTTP login
- [ECK] In 3.3.x, the `stackconfigpolicy-controller` repeatedly attempted GETs on resources outside managedNamespaces every 17 minutes, emitting reconcile errors. Worked around by setting `managedNamespaces: []` (cluster-wide watch); RBAC was already cluster-scoped (`createClusterScopedResources:true`) so no permission delta

- [ECK] Fluentd has no official ES9 image variant, forcing use of the ES8 variant (v1.19.2-debian-elasticsearch8-1.4). Validated compatibility with ES 9's _type removal via fluent-plugin-elasticsearch 5.4.4 and suppress_type_name:true, then verified the fluent-bit → fluentd → ES path end-to-end by injecting an active marker log into the NFS PVC

Results

- 90% log search time reduction (individual Pod access → single Kibana interface)
- Achieved 70% MTTR improvement through unified logging system, maintaining 99.9% service availability and minimizing potential revenue loss risk from incidents
- Real-time processing of 1GB daily log data with 90-day retention
- 70% log collector memory reduction through EFK migration (Logstash JVM 1GB → Fluent Bit 50MB)
- Unified Helm-based logging pipeline management with automated chart upgrades
- ECK Operator migration eliminated manual operations for TLS certificates, account passwords, and StatefulSet upgrades. Rolling upgrades executable with a single-line CR spec.version change
- Completed Elastic Stack 8.5.1 → 9.0.0 major upgrade (closed a 2-year update gap)
- With ECK Operator's secureMode + ServiceMonitor, controller-runtime-based metrics are automatically registered in kube-prometheus-stack, establishing visibility into Operator internals

Tech Stack

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 5 internal tools to automate repetitive manual tasks and boost development team productivity.

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Problem

Manual APK delivery to QA team after build completion caused sharing delays and version confusion

Solution

Developed Python bot that auto-sends build completion messages with download QR codes to Slack when Jenkins APK builds finish, deployed on Kubernetes

Results

90% QA team delivery time reduction, eliminated version confusion with QR code-based instant install, automated build history logging

Tech Stack

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.08)

Problem

Dual work of writing technical docs in GitLab markdown then manually uploading to Google Drive

Solution

Integrated GitLab Webhook with Google Drive API for auto-sync on push. Python converts markdown → Google Docs for upload

Results

80% document management time reduction (10min → 2min per doc), unified GitLab as Single Source of Truth

Tech Stack

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. Internal LLM Service Deployment (4 weeks, 2026.01)

Problem

Growing LLM demand from dev team, cost burden and internal data security concerns with external SaaS

Solution

Deployed Ollama and Open WebUI on Kubernetes on on-premises GPU server (RTX 3060). Configured model storage with NFS

Results

Serving 15 developers with an average of 100 queries/day for code review, document summarization, and debugging assistance. Saved ~\$100/month by replacing external AI SaaS subscriptions.

100% compliance with internal security guidelines and eliminated external data leakage risks

Tech Stack

Ollama, Open WebUI, Kubernetes, NFS, GPU (RTX 3060)

4-4. Git Repository Mirroring Tool Development (4 weeks, 2026.02)

Problem

Manual source code synchronization across AWS CodeCommit, GitLab, GitHub caused omissions and delays

Solution

Developed Go-based bidirectional Git mirroring tool (git-bridge). Supports SQS polling (CodeCommit) + HTTP Webhook (GitLab/GitHub) event sources with incremental sync (git fetch) and loop detection.

Deployed on Kubernetes with Slack notification integration

Results

Processes an average of 30 sync events/day across 10 repositories, fully automating multi-platform source sync and eliminating manual mirroring; open-sourced and utilized alongside GitHub Actions (multi-git-mirror)

Tech Stack

Go, AWS SQS, GitLab Webhook, GitHub Webhook, Kubernetes, Docker

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Problem

The previously used halverneus/static-file-server:v1.8.11 provided only basic file serving, lacking operational features such as directory listing UI, file preview, search/filter, and batch download. Demands for improved UX in QA APK distribution and internal file sharing persisted, while the upstream chart maintenance had stalled.

Solution

Designed and developed a lightweight Go-based static file server (static-file-server) in-house and operated a Helm repository on GitHub Pages (<https://somaz94.github.io/static-file-server/helm-repo>) to establish a deployment standard.

Deployed to Kubernetes with an in-house Helm chart, connected to an NFS StorageClass (nfs-client-nopath, Retain, 5Gi) for stable long-term artifact hosting. The legacy halverneus/static-file-server:v1.8.11 deployment was moved to _deprecated/static-file-server/ and fully replaced.

Key Features

- Dark-mode directory listing UI (grid/list view switching, keyboard navigation)
- File preview (image / video / audio / PDF / text · code)
- Search / filter + URL hash sharing, multi-select + ZIP batch download
- Gzip compression, Prometheus metrics, structured JSON logging, health check
- Automatic APK file Content-Type handling (via ingress annotation)

Results

Released as open source under Apache-2.0 (operating Docker Hub and GitHub Pages Helm repo). Handles an average of 30 downloads/day and 5 APK distribution builds/week (daily build), unifying various internal file-sharing needs — APK distribution, document sharing, build artifact delivery — into a single tool, and standardizing deployment / upgrade / rollback via the in-house Helm chart.

Eliminating the upstream dependency enabled managing security patches and new features on an in-house cycle.

Tech Stack

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Monitoring design, upgrade automation, Helm management framework, NGF migration, 2 open-source Helm chart releases)
2025.12 ~ Present

To improve cluster visibility, operational stability, and network modernization, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API. Released both the CR management structure derived from the NGF migration and the ECK Operator-based logging stack as separate open-source Helm charts.

5-1. Monitoring & Alerting Enhancement (2026.01 ~ Present)

Problem

The default kube-prometheus-stack install lacked custom alerts and dashboards across the infrastructure/DB/network layers, and was missing metric collection for physical servers, VMs, and the Cilium CNI — making proactive failure detection difficult.

Solution

Designed 18 custom alert rules (9 Node, 2 Pod, 4 Cilium) based on kube-prometheus-stack and produced 10 custom Grafana dashboards. Integrated MySQL/Redis/Elasticsearch/PostgreSQL exporters and deployed node-exporter on 4 physical servers and VMs via Ansible. Collected Cilium CNI agent/operator metrics via kubernetes_sd_configs and configured severity-based Slack routing with inhibit rules in Alertmanager to improve alert quality.

Results

Built proactive failure detection across infrastructure/DB/network layers with 18 custom alert rules and 10 dashboards, and extended unified observability coverage to physical servers and VMs. Severity-based routing and inhibit rules removed duplicate alerts, improving on-call response quality.

Tech Stack

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ 2026.04)

Problem

Kubespray-based K8s upgrades were performed manually, making pre-flight validation, backup, and compatibility checks inconsistent, and with no versioning standard across the 18 infrastructure Helm charts, upgrade scripts drifted per chart. Kubernetes certificates expired every 6 months, exposing the API Server to outage risk.

Solution

Built a Kubespray version management automation script (pre-flight validation, inventory backup, version compatibility check, breaking change detection) and a 9-step post-upgrade health check script (node/Pod/DNS/cert/etcd/API Server).

Built an upgrade script framework across all 18 infrastructure Helm charts (14 external + 4 local) along with a synchronizer that manages four canonical templates as a single source of truth (check mode detects drift in CI, apply mode propagates in bulk).

The automated Kubernetes certificate renewal script supports a stepwise flow (verify → backup → renew → kubelet/apiserver restart → kubeconfig update → final verification) with rollback, scheduled via crontab on 10 nodes every 6 months at 3 AM.

Results

The 9-step automated health check reduced post-upgrade verification time by 90%, while the upgrade framework + canonical template synchronizer across 18 Helm charts standardized infrastructure version management and enabled CI-based drift detection of script bodies. The automated certificate renewal script with crontab scheduling eliminated API Server outage risk from certificate expiration across 10 nodes.

Tech Stack

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Problem

On the on-premises cluster, 11 ingress-nginx controller instances (default + public-a~j with pinned MetalLB LoadBalancer IPs 10.10.10.55-58, 62-68) were duplicating Deployment/Service/CRD/RBAC 11 times to provide ingressClassName-based multi-tenancy. With Kubernetes Gateway API reaching v1.2 GA and ingress-nginx slated for maintenance mode, the network controller had to be migrated to its Gateway API-based successor.

Solution

Adopted the official successor NGINX Gateway Fabric (NGF) 2.x to simplify the topology into a single control-plane with 11 Gateway CRs, preserving the MetalLB IPs so DNS/firewall stayed unchanged via parallel deployment + gradual cutover. Completed the Phase 0~7+ pipeline (MetalLB pool expansion → NGF install → 5 observability apps → HTTP-only / 9 ApplicationSet apps → HTTPS-enforced apps (Harbor, Vaultwarden) → IP-swap cutover 11/11 → Ingress cleanup) within two days.

Mapped 10 ingress-nginx annotations 1:1 to Gateway API + NGF CRDs (HTTPRoute filters, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy) and unified the HTTPRoute chart values schema across four charts for batch migration of 9 ApplicationSet apps. Consolidated two per-app TLS Secrets (harbor-tls, vaultwarden-tls) into a single self-signed wildcard certificate (*.concrit.us, 10-year validity).

Troubleshooting

- IPs remained held by MetalLB even after scaling Pods to 0 when externalTrafficPolicy was Cluster with a pinned loadBalancerIP, blocking the swap to NGF. Resolved by patching the Service to ClusterIP and removing the loadBalancerIP field as a new step in the cutover.sh script to force release
- Lingering ingress-nginx admission ValidatingWebhookConfigurations blocked Ingress UPDATE/DELETE with 2-minute timeouts when the endpoint Service had zero Pods. Added preemptive VWC deletion to the cutover step, which also eliminated foregroundDeletion finalizer stuck issues

- NGF chart's default externalTrafficPolicy: Local caused 2-minute timeouts on internal Pod → public domain → LB IP paths (dataplane was single-replica, so most nodes missed locally). Explicitly set externalTrafficPolicy: Cluster across 11 NginxProxy CRs to match the prior ingress-nginx policy, fixing CI generator stage regressions

Results

Collapsed 11 controller instances into a single NGF control-plane + 11 Gateway CRs — eliminating CRD/RBAC/Deployment duplication — and completed 11 class cutovers with zero incidents (30-60s actual downtime per class). Standardized the ApplicationSet batch migration flow by unifying HTTPRoute chart values schema across 4 charts, consolidated two self-signed certs into a single wildcard (10-year validity) cutting manual renewal overhead by 50%, and minimized the migration window by completing the full Phase 0~7+ pipeline within two days.

Tech Stack

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. nginx-gateway-cr Open-Source Helm Chart Release (2026.04)

Problem

The NGF upstream chart installs only the controller and CRDs, and does not provide the tenant-level resources needed for actual traffic paths (Gateway, NginxProxy, ReferenceGrant, ServiceMonitor, PodMonitor). In NGF 2.x the controller /metrics endpoint is exposed on the Pod rather than the Service, making traditional ServiceMonitor scraping unsuitable, and no reusable community chart covered this gap.

Solution

Extracted the local CR chart structure produced during the NGF migration, refined it, and released it as the open-source nginx-gateway-cr — a reusable Helm chart that deploys all five tenant-level resources.

Includes a multi-Gateway-friendly schema (gateways[] array with per-gateway overrides), PodMonitor templates recommended for NGF 2.x (controller + dataplane), and values.schema.json-based input validation, and is distributed via both an OCI registry and a traditional Helm repo to cover both install paths.

Results

Registered under Apache-2.0 in the ArtifactHub networking category, establishing a reusable community chart artifact for similar NGF environments. Achieved dual utilization by reusing the in-house NGF migration output — managed via the internal helmfile pipeline and simultaneously distributed externally as open source.

Tech Stack

Helm Chart, Gateway API, NGINX Gateway Fabric, Prometheus Operator, OCI Registry, ArtifactHub

5-5. ECK Operator-based Elasticsearch / Kibana Helm Chart Release (2026.04)

Problem

Even after adopting the ECK Operator, the Elasticsearch/Kibana configuration remained in a local Helm chart tightly coupled to internal storageClass names and per-cluster CR names, blocking external reuse. We needed a single public chart that bundled the ECK Operator resources, along with a migration path that could move an already-running cluster from the local chart to the public chart without data loss.

Solution

Designed two new charts — charts/elasticsearch-eck (11 ES templates) + charts/kibana-eck (10 Kibana templates) — that deploy the CRs, Secrets, HTTPRoute, Ingress, BackendTLSPolicy, ServiceMonitor, and NetworkPolicy needed to run Elasticsearch/Kibana from a single set of values.

Added values.schema.json (draft-07) validation for HA options, storage, and resource inputs; documented Minimal/HA presets and per-environment storageClass mapping in the README; and published simultaneously across three channels — OCI (ghcr.io/somaz94/charts), a gh-pages Helm repo, and ArtifactHub (monitoring-logging category, Apache-2.0). Also authored a new canonical scripts/upgrade-sync/templates/oci-chart-version.sh template to tighten the internal OCI chart-version tracking procedure.

Results

During the mgmt cluster's switch from the local chart to the OCI chart (elasticsearch-eck / kibana-eck 0.1.1), every CR / PVC / Secret name was preserved — cluster_uuid stayed unchanged and the migration completed with zero Elasticsearch pod restarts and a single Kibana rolling restart, i.e. no data loss.

HA rolling scenarios passed 3/3 on a local kind cluster, end-to-end install on the real cluster reached Ready/green in 71s for Elasticsearch and 57s for Kibana, and publishing on ArtifactHub added the second reusable public chart after nginx-gateway-cr — extending the dual-use model (internal helmfile pipeline + external open-source distribution) to the logging stack.

Tech Stack

Helm Chart, ECK Operator, Elasticsearch, Kibana, Gateway API, OCI Registry, ArtifactHub, Prometheus Operator, Kubernetes

Project 6: Infrastructure Security System

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Building security infrastructure for safe management of sensitive information required for infrastructure operations.

6-1. Vaultwarden Password Management System (2026.04)

Deployed Vaultwarden (Bitwarden-compatible server) on Kubernetes via Helm with GitLab SSO (OpenID Connect) integration. Managed self-signed TLS certificates through Helm extraObjects, configured organization/collection-based team access control.

Built automated backup CronJob (daily SQLite dumps, 30-day retention) and interactive restore script for data reliability.

Results

- Migrated 70 users and 50 secrets into centralized management, eliminating security risks and improving handover efficiency
- Immediate login with existing company GitLab accounts via SSO integration, removing separate account provisioning/deprovisioning overhead and improving account management efficiency

Tech Stack

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game and blockchain startup, leading the large-scale cloud migration project from AWS to GCP.

Transitioned to leverage GCP Startup Program cost optimization and GCP's global network performance, successfully completing the full service migration. Operated a dual source management system with GitLab for on-premises and GitHub for production environments.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Reconstruction

Contribution: Infra architecture design, migration strategy & execution lead 70%, CI/CD pipeline construction 100% 7 months (2023.05 ~ 2023.12)

Problem

AWS costs were continuously rising (\$10,000+/month), particularly NAT Gateway and data transfer costs. GCP's global load balancing was needed for multi-region game services, and the existing Jenkins-based deployment system had complex configuration management with no deployment history tracking.

Approach

Established a 3-phase migration strategy executed sequentially. Phase 1: Built and validated dev environment on GCP.

Phase 2: Migrated QA environment. Phase 3: Migrated production.

Designed the network on Shared VPC (Host Project / Service Project separation) and switched GitHub Actions' GCP auth to Workload Identity Federation, eliminating service-account key issuance/rotation overhead in favor of short-lived OIDC tokens. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for pre-validation, then performed the final NS swap during a single scheduled maintenance window for zero-downtime cutover. Worked around Filestore's 2.5TB SSD minimum cost by self-hosting an NFS server on Compute Engine (pd-balanced 1TB), minimizing operational cost. Configured Cloud Armor with region blocking + IP allow-list WAF policy and split the Cloud CDN URL Map into HTTP / HTTPS variants to enforce HTTP → HTTPS redirect alongside static-asset caching.

Codified GCP infrastructure with Terraform (100% excluding IAM), then rebuilt the GitOps CI/CD pipeline combining GitLab CI and ArgoCD.

Standardized environment configs with Helm Charts and established Git commit-based deployment history management.

Troubleshooting

- Traffic routing issues during AWS ALB to GCP Global LB transition due to health check and backend service configuration differences. Analyzed GCP NEG (Network Endpoint Group) structure and reconfigured for GKE workloads
- Data corruption during AWS RDS to Cloud SQL migration due to character set differences. Resolved by writing pre-validation scripts and implementing step-by-step data integrity check processes
- Network design issues due to AWS vs GCP subnet model differences (AZ-based vs region-based). Redesigned CIDR blocks and converted firewall rules to GCP tag-based approach
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response. Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig to restore external ingress traffic
- Game client file downloads failed after the Cloud CDN migration due to a missing HTTP → HTTPS redirect. Resolved by splitting the URL Map into HTTP / HTTPS variants (default_url_redirect.https_redirect=true) and mapping the HTTP forwarding rule to a dedicated port-80 target HTTP proxy

Results

- 50% cloud cost reduction (\$10,000 → \$5,000/month), ~\$60,000 annual savings

- Completed full service migration via the 3-phase strategy. Minimized downtime with resource-copy based migration, and executed the final DNS cutover during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Established IaC management for entire GCP infrastructure with Terraform (ensured code quality with TFLint/Checkov static analysis and Terratest unit testing)
- 50% deployment time reduction (40min → 20min), 95% rollback time decrease (30min → 1-2min)
- 3x weekly deployments (2 → 6), 100% deployment history tracking via Git commits
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions, transitioning to a short-lived token security model

Tech Stack

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, with analysts spending 2-3 hours daily on data collection and frequent data gaps due to human errors.

Approach

Built a GCP-based pipeline that loads multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery and automatically refreshes analyst-facing Google Sheets. MongoDB was ingested via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection feature, and GA / Dune via their respective APIs (Dune with a separate API key), normalized into a consistent index schema.

Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, and Cloud Scheduler triggers them on Daily (previous-day data) / Monthly (previous-month data) cadences. A separate dedup Cloud Function runs periodically to keep BigQuery data consistent. The full infrastructure (BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM bindings, service accounts) is managed as Terraform IaC for reproducible environments and change tracking.

Results

- 95% data collection automation (eliminated 2-3h daily manual work)
- 0% data gap rate through automation eliminating human errors
- Terraform IaC for the full infra enabled rebuilding the same pipeline in another GCP project within 30 minutes, with 100% change history in Git

Tech Stack

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Dune API, MongoDB, CloudSQL, Docker

Project 3: PLG Stack Monitoring System

Contribution 100% (System design & implementation) 1 month (2024.04)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive incident response with excessive time spent on root cause analysis.

Approach

Built a PLG Stack collecting metrics with Prometheus and logs with Loki, unified visualization through Grafana dashboards. Real-time monitoring of CPU/memory/network usage and application logs, with automatic Slack alerts on threshold breaches.

Results

- 80% proactive incident detection rate through threshold-based alerts
- 60% incident response time reduction through real-time monitoring and alerting
- Service availability improved from 99.5% to 99.9%
- Reduced operations team fatigue and improved actual incident detection accuracy through SLI/SLO establishment and Alertmanager alarm noise optimization

Tech Stack

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

Responsible for building PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and providing technical support. Designed and built on-premises cloud infrastructure for financial and public sector clients, and conducted training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

Designed Kubernetes clusters in HA (High Availability) configuration per client requirements and built virtual machine management environments with OpenStack. Provided unified block/file/object storage through Ceph and conducted parallel infrastructure operations training for client teams.

Results

- Successfully built and delivered PaaS solutions to 5 clients
- Achieved 99.9% availability with Kubernetes cluster HA configuration
- Ansible-based server provisioning and configuration management automation reducing per-client deployment time and eliminating configuration drift issues

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

Designed a step-by-step training curriculum from Kubernetes basics to cluster operations and troubleshooting, with hands-on lab environments for practice-oriented training. Also conducted parallel training on OpenStack and Ceph storage operations to ensure full PaaS stack operational capability.

Results

- Operated the curriculum with an average of 10 attendees per session and 4 hours per session, establishing DP's autonomous Kubernetes operations system
- Reduced client technical inquiries post-training, achieved self-incident response capability
- Standardized training materials reused for subsequent client trainings

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

Project 3: Kubernetes Certificate Auto-renewal Process Improvement

Contribution 100% (Solo) 2 weeks (2022.11)

Problem

Kubernetes cluster certificates expired annually, requiring ~5 hours of renewal work per client per year, with service outage risks upon expiration.

Approach

Analyzed the Kubernetes certificate management process and modified kubeadm settings to extend certificate validity from 1 year to 10 years. Developed automation scripts applicable to existing clusters and deployed to all clients.

Results

- 10x certificate renewal cycle extension (annual → once per 10 years)
- ~50 hours annual operations savings across 10 clients
- Eliminated outage risks from certificate expiration

Tech Stack

Kubernetes, kubeadm, Bash Script, OpenSSL